

Politecnico di Torino

Digital Signal Processing and Optical Transmission Lab

DSP Report - Gruppo 11

Arianna Di Giusto (352119) **Lorenzo Pellegrino (354773)**
Andrei Sgobio Plugariu (358919)

Politecnico di Torino – January 19, 2026

Contents

1 Laboratory on audio filtering **1**
1.1 Task 1: Floating Point Filter Structures . . . 1
1.1.1 FIR Filtering: Linear vs. Circular Buffer . . . 1
1.1.2 IIR Biquad Filtering: Direct Form I vs. Direct Form II . . . 1
1.2 Task 2: Fixed Point Implementation . . . 2
1.2.1 FIR Fixed-Point Implementation (Q0.15) . . . 2
1.2.2 IIR Fixed-Point Implementation (Q3.12) . . . 3
1.3 Task 3: Real-time Filtering . . . 3
1.3.1 FIR fixed point filter in real time operation (f=24 kHz) . . . 4
1.3.2 IIR fixed point filter in real time operation (f=24 kHz) . . . 5
1.4 FIR vs IIR fixed point real time at f=48 kHz . . . 7

2 Laboratory on audio compression **11**
2.1 Task 1: Test of the audio codec . . . 11
2.2 Task 2: Real-time audio coding . . . 12
2.2.1 Silence and Sine Wave Analysis . . . 12
2.2.2 Adaptive Rice Parameter on Real Audio . . . 12

Chapter 1

Laboratory on audio filtering

1.1 Task 1: Floating Point Filter Structures

The objective of this first task was to implement and compare different filter structures using floating-point arithmetic. Since the DSP board lacks a hardware Floating Point Unit, the compiler emulates floating-point operations through fixed-point software routines.

1.1.1 FIR Filtering: Linear vs. Circular Buffer

This section compares the CPU performance of a Finite Impulse Response (FIR) filter implemented using a standard linear buffer versus a circular buffer. We implemented two versions to assess the computational overhead:

- **Linear Buffer:** Requires explicit data shifting of $L - 1$ elements after each sample processing, introducing significant overhead ($O(N)$ operations).
- **Circular Buffer:** Uses a pointer wrapping mechanism to manage the delay line, eliminating the need for data shifting.

Performance and Accuracy Results The error between the fir asm and the fir linear strictly depends on the used coefficients. By using hf coefficients the filter is not correctly represented since we introduce some rounding error, if the coefficients are changed to hfq the result of the error was way more accurate, equal to 0.084 (very similar to 1/12). Moreover, the error between the linear and circular representation was negligible (i.e., $\approx 10^{-8}$). This is because both algorithms perform the exact same floating-point arithmetic operations, merely accessing the state buffer memory in a different order.
The CPU cycles for both implementations were measured using the Code Composer Studio profiling tool for each call to the filter routine (processing $N = 256$ samples. The results are summarized in Table 1.1.

Table 1.1: Average CPU Cycles per call (N=256) for FIR Floating-Point Implementations.

Optimization	Linear Buffer (Cycles)	Circular Buffer (Cycles)	Improvement (Circular)
Disabled	6,429,954	6,025,652	6.29%
-O0	6,241,794	5,961,469	4.49%
-O1	6,188,280	5,918,692	4.36%
-O2	5,704,525	5,686,689	0.31%
-O3	5,737,589	5,719,786	0.31%

Analysis of Results The data in Table 1.1 shows two distinct behaviors. At low optimization levels (Disabled, -O0, -O1), the circular buffer is demonstrably more efficient (4-6% fewer cycles). The overhead of the data-shifting loop in the linear buffer is a significant performance penalty. However, at the highest optimization level (-O2 and -O3, which are similar), the performance difference becomes negligible (0.31%). This suggests that the compiler successfully identified the memory-shifting pattern in the linear buffer and optimized it using highly efficient block-move instructions, making its performance comparable to the pointer-based logic of the circular buffer.

1.1.2 IIR Biquad Filtering: Direct Form I vs. Direct Form II

This section compares the CPU performance of an IIR biquad filter. We implemented:

- **Direct Form I:** Requires two separate delay buffers (one for input $x[n]$, one for output $y[n]$).
- **Direct Form II:** Combines the delay lines into a single shared state buffer, theoretically reducing memory accesses.

Performance and Accuracy Results By building and debugging the project, it was confirmed that the Mean Squared Error (MSE) between the two forms is negligible (5.18×10^{-8}), confirming their mathematical equivalence in floating-point. The performance results are shown in Table 1.2.

Analysis of Results The data in Table 1.2 shows that the Direct Form II implementation is consistently more efficient than Direct Form I across all optimization levels (approx. 2-2.6% improvement). Unlike the FIR case, this advantage is structural. DF-II requires fewer memory read/write operations to manage its state (one buffer vs. two). This algorithmic advantage cannot be completely optimized away by the compiler, resulting in a consistent performance gain.

Table 1.2: Average CPU Cycles per call (N=256) for IIR Biquad Floating-Point Implementations.

Optimization	Direct Form I (Cycles)	Direct Form II (Cycles)	Improvement (DF-II)
Disabled	6,016,660	5,866,839	2.49%
-O0	5,923,808	5,812,132	1.88%
-O1	5,817,337	5,672,114	2.50%
-O2 / -O3	5,819,998	5,668,002	2.61%

1.2 Task 2: Fixed Point Implementation

The objective of this task was to port the floating-point filter structures from Task 1 to fixed-point arithmetic, leveraging the DSP's native 16-bit architecture.

1.2.1 FIR Fixed-Point Implementation (Q0.15)

The FIR filters (linear and circular) were ported to fixed-point. The input/output samples and filter coefficients (vector 'h') were all represented in **Q0.15 format**.

Implementation The core logic involves replacing floating-point multiplication with 16-bit fixed-point multiplication. This requires intermediate storage in a 32-bit accumulator (long) and proper scaling (right-shifting by 15 with rounding) to return to the Q0.15 format. Both linear and circular buffer approaches were implemented following these principles.

Accuracy Analysis The Mean Squared Error was measured between the fixed-point implementations and the floating-point reference, using two different sets of float coefficients.

Table 1.3: FIR Fixed-Point vs. Floating-Point, MSE.

Fixed-Point Version	Fix vs. Float (coeffs 'hf')	Fix vs. Float (coeffs 'hfq')
FIR Linear (Fixed)	0.350 714 417	0.083 736 024 8
FIR Circular (Fixed)	0.350 714 417	0.083 736 024 8

Analysis:

- **vs. 'hf':** The error (0.35) is larger because it includes two components: the **coefficient quantization error** and the **arithmetic rounding error** from the Q0.15 multiplications.
- **vs. 'hfq':** The 'hfq' coefficients are the floating-point equivalents of the quantized 'h' coefficients. Therefore, the coefficient error is zero, and the remaining MSE (≈ 0.084) represents **only the arithmetic rounding error**. This error is consistent with the theoretical quantization noise model ($\sigma_q^2 = \Delta^2/12$).

CPU Cycles Analysis At this point the CPU Cycles taken to process were again measured with different optimization levels.

Table 1.4: Average CPU Cycles per call for FIR Fixed-Point Implementations

Implementation	Disabled	-O0	-O1	-O2	-O3
FIR Linear	538,130	354,320	354,573	33,048	33,048
FIR Circular	317,787	192,583	176,448	25,543	25,543
FIR ASM	13,858	13,858	13,858	13,858	13,858

Analysis:

- **Optimization Impact:** The C compiler's optimization (-O2) has a *massive* impact, reducing cycle counts by over 90%. This shows the compiler is highly effective at optimizing native fixed-point C code.
- **Linear vs. Circular:** At all levels, the circular buffer implementation is significantly faster (e.g., $\approx$ 23% faster at -O2). This confirms its theoretical advantage: it eliminates the data-shifting required by the linear buffer.
- **Fixed vs. ASM:** The 'fir_asm' function is the fastest even compared with the best optimized fixed FIR. The ASM function is specifically coded to exploit the DSP's architecture, likely using the hardware circular buffer addressing mode and ensuring the MAC pipeline is always full.

1.2.2 IIR Fixed-Point Implementation (Q3.12)

The IIR biquad filters (DF-I and DF-II) were ported to fixed-point using **Q3.12 format** for coefficients and **Q0.15** for I/O data.

Implementation This conversion is more complex due to the mix of formats (Q0.15 data, Q3.12 coefficients) and the recursive loop.

- **DF-I:** Implemented as a cascade of a Q3.12 FIR section and a Q3.12 IIR section, requiring two 16-bit state buffers.
- **DF-II:** Implemented using a single shared 16-bit state buffer. A critical detail is scaling the Q0.15 input $x[n]$ by 1.0 in Q3.12 format (i.e., multiplying by 2^{12}) before the first summation. This aligns the data to the Q3.12 format of the state buffer.

Accuracy (MSE) and Overflow Analysis The MSE was measured for both IIR forms using two different test signals ('samples.h' and 'samples2.h') and two different coefficients vector ('af, bf' and 'afq, bfq').

Table 1.5: IIR Fixed-Point vs. Float (coeffs 'af, bf') MSE.

Signal	MSE (DF-I)	MSE (DF-II)
samples.h	2.946 340 32	816 734.062
samples2.h	1.197 755 93	0.396 489 918

Table 1.6: IIR Fixed-Point vs. Float (coeffs 'afq, bfq') MSE.

Signal	MSE (DF-I)	MSE (DF-II)
samples.h	0.808 875 144	816 049.625
samples2.h	0.884 770 215	0.087 512 314 3

Analysis:

- **Case 1: 'samples.h' (Overflow):** Direct Form II is subject to an **internal overflow** leading to a value of over 816k. DF-II calculates the recursive (all-pole) section first. This section can have high gain, causing the intermediate values in the shared state buffer to exceed the limits of the 16-bit 'short' or 32-bit 'long' accumulator. Direct Form I computes the FIR (all-zero) section first, which often attenuates the signal *before* it enters the recursive loop, preventing overflow.
- **Case 2: 'samples2.h' (No Overflow):** This signal does not cause overflow, probably due to lower amplitude. Here, DF-II is significantly **more accurate** (MSE ≈ 0.0875) than DF-I (MSE ≈ 0.885). This is due to differing **quantization noise** properties. In DF-I, the rounding error from the FIR section is *fed back* into the recursive loop, where it is amplified. In DF-II, the main rounding point occurs *after* the recursive loop.

CPU Cycles Analysis The fixed-point IIR implementations were profiled against their floating point version.

Table 1.7: Average CPU Cycles per call for IIR Implementations.

Implementation	Disabled	-O0	-O1	-O2	-O3
DF-I (float)	593,272	582,771	570,665	567,320	567,320
DF-II (float)	578,885	572,126	556,982	552,889	552,889
DF-I (fix)	44,307	42,774	25,880	8473	8473
DF-II (fix)	40,211	33,043	23,316	8213	8213

Analysis:

- **Fixed vs. Float:** The performance gain is enormous. The fixed-point optimized versions are $\approx$ **65 – 70× faster** than the floating-point versions. This confirms that for this DSP, fixed-point is the only viable option for high-performance real-time processing.
- **DF-I vs. DF-II (Fixed):** Direct Form II is marginally faster ($\approx 3\%$) than Direct Form I. This is because it uses a single, shared state buffer, resulting in fewer memory accesses and data movements compared to DF-I's two separate buffers.

1.3 Task 3: Real-time Filtering

In this task, the goal is to validate the fixed-point FIR and IIR filters developed in the task 2 by running them in real time on the DSP. To achieve this, the `audio_loop` project implements a mechanism based on a DMA double buffering. The DMA alternatively sends samples to either buffer 1 or buffer 2, triggering an interrupt when a buffer is full. The DSP must process the free buffer within a time $T_{\text{proc}} < \frac{N}{F_s}$ to ensure continuous acquisition without data loss.

1.3.1 FIR fixed point filter in real time operation (f=24 kHz)

Implementation The fixed-point FIR filter implemented in Task 2 was integrated into the real-time audio loop project. The DSP executes the filtering inside the main loop whenever the flags `RunFilterForL` and `RunFilterForR` are raised by the DMA interrupt. Since the audio sample are continuously acquired through the ADC, there is no break point in the code.

If the `enable_loop` flag is set to 1, the input is copied directly to the output, thus the filter is not applied; otherwise, if set to 0, the FIR (linear or circular) is applied in real time.

Table 1.8: Measured real-time samples_per_sec for different optimization levels and FIR filter applied (enable_loop = 0).

FIR Structure	Optimization Level	samples_per_sec	Real-time
Linear	-O2	47974	Yes
Circular	-O2	47974	Yes
Linear	Disabled	11878	No
Circular	Disabled	20531	No

Performance and Accuracy Results The variable `samples_per_sec` was used to understand if the DSP was able to process all the incoming audio buffers from both channels (left and right). As we can see from table 1.8, when the compiler optimization was set to `-O2`, both the linear and circular FIR filters achieved approximately 48,000 samples/s, which corresponds to the theoretical expectations for a sampling frequency of the ADC set at 24 kHz per channel across two channels. This is because each block contains 256 samples (as defined in the code "XMIT_BUFF_SIZE = 256") for the left channel and 256 for the right channel, for a total of 512 samples per block. Thus the total number that the DSP should process is $48,000$ samples per second across both channels.

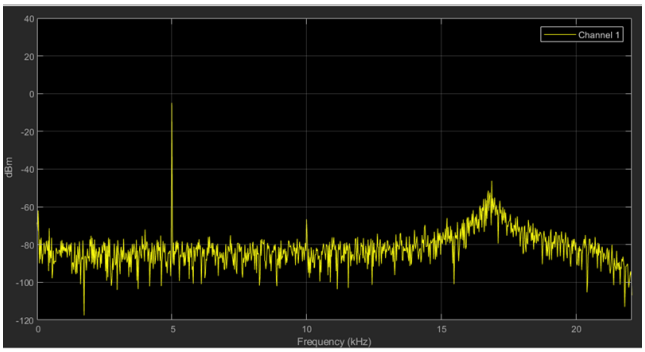
When optimizations were disabled, the sample_per_sec dropped significantly. The linear fixed-point FIR reached only 11,878 samples/s, while the circular buffer version performed slightly better at 20,531 samples/s. In both cases, the DSP was unable to process all DMA buffers before the next ones arrived, causing missed blocks and a reduction in the effective processing rate. These results are consistent with those found in task 1 and 2, where the linear filter is worst because of the explicit shifting of the delay line. In the table, only results with optimization level Disabled and -O2 are shown, because the ones with less optimization level are able to reach the maximum number of samples per second too.

Verification with some audio filters from Matlab In this section, we tested several signals generated from MATLAB to verify that the real-time implementation matches the expected behavior. The optimization level was always set to `-O2` and we exploit both FIR linear and circular implementation, even though in the report only the former is shown since the latter behaved in the same manner.

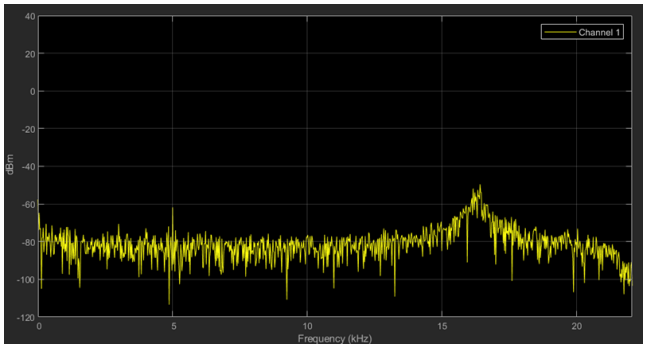
It is important to notice that MATLAB signals are generated with a frequency of 48kHz, thus the usable bandwidth of the signals is around 24kHz. We chose a sampling frequency for the ADC of F_s = 24kHz, limiting the Nyquist bandwidth of the real-time system to 12kHz. Therefore, any spectral content above 12kHz will show aliasing when acquired by the DSP. Nevertheless, the aliasing that were shown in the graph were introduced not by the DSP (that has its own anti-aliasing filter), but by the PC sound card.

The following behaviors were observed:

- **Tone 5k, 30, 48k**
  A tone signal corresponds to a single frequency component, producing a single fixed spectral peak (in this case at 5kHz).



(a) Tone with enable_loop=1

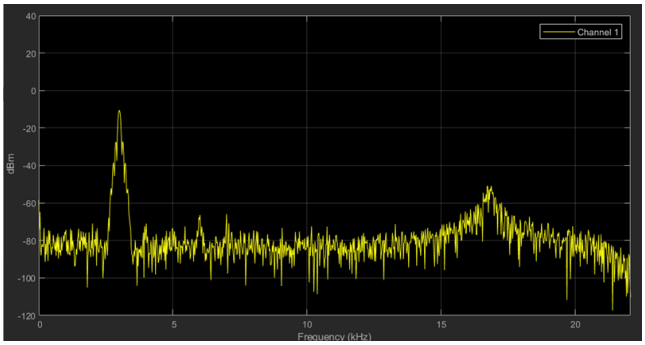


(b) Tone with enable_loop=0

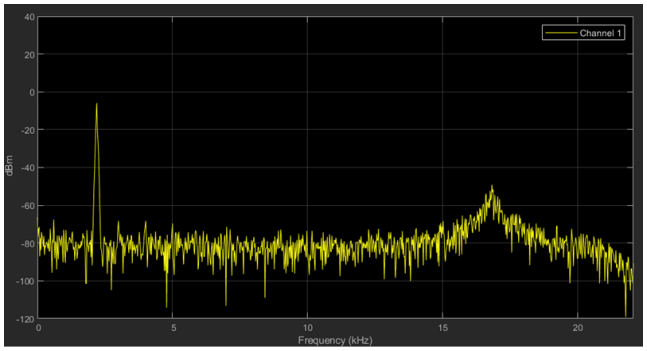
Figure 1.1: Tone 5k, 30, 48k

With `enable_loop = 1`, the DSP bypasses the FIR filter and the output spectrum shows a clean spectral peak at 5kHz. When `enable_loop = 0`, the FIR filter is active: from figure 1.3b, we can see that the cutoff frequency is around 3kHz, thus the peak in Figure 1.1a is well attenuated, while the rest of the spectrum remains unchanged.

- **Chirp 48k, 12k, 1k**
  A chirp signal is a frequency-swept signal: its instantaneous frequency varies over time. In the frequency domain, this appears as a moving peak.



(a) Chirp with enable_loop=1

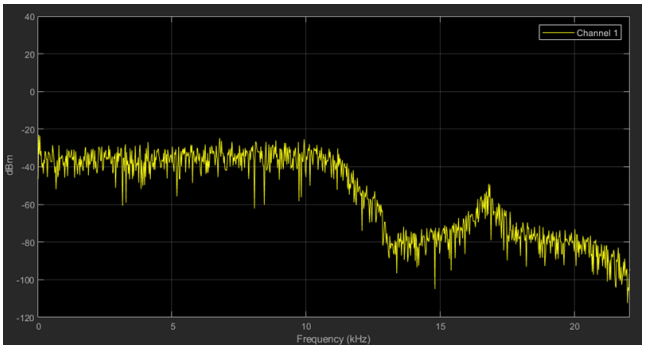


(b) Chirp with enable_loop=0

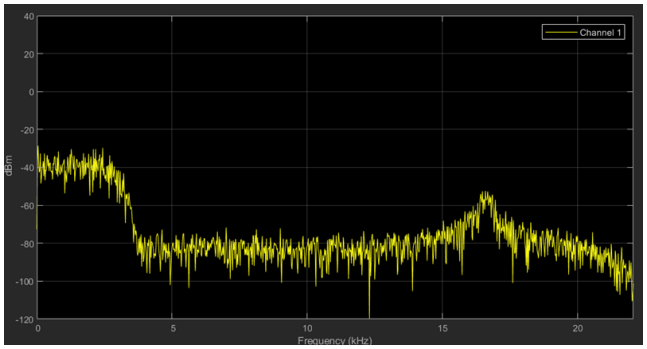
Figure 1.2: Chirp 48k, 12k, 1k

With `enable_loop = 1`, the chirp is reproduced up to 12kHz, above which there is aliasing. When the FIR filter is enabled (`enable_loop = 0`), only the frequency below 3kHz are preserved, while the higher ones are attenuated. To better understand the aliasing phenomenon, we tried to set the sampling frequency to F_s = 10kHz, therefore the Nyquist limit became 5kHz, while the chirp generated in MATLAB still sweeps up to 12kHz. It this situation we saw the spectral peak rose until around 5kHz and then bounced back towards the lower frequencies. We also tried to enable the FIR filter without seeing any significant effect, because this happened before the cutoff frequency.

- **White noise 48k, 30**
  A white noise signal appears as a flat spectrum up to 12KHz, after which there will be aliasing.



(a) White noise with enable_loop=1



(b) White noise with enable_loop=0

Figure 1.3: White noise 48k, 30

With `enable_loop = 1`, the FIR filter is disabled and, as shown in Figure 1.3a, the output of the DSP has a flat spectrum up to 12kHz. When the FIR filter is active (`enable_loop = 0`), we can clearly see that the noise signal is attenuated at a frequency of 3kHz, that is exactly the cutoff frequency of the low-pass filter, as already exploit while dealing with the tone signal. This is coherent with the theory, since the digital filter works with a frequency that is between 0 and 1/2 of the sampling frequency, so $3/24 = 1/8$.

1.3.2 IIR fixed point filter in real time operation (f=24 kHz)

Implementation In this section, we exploited the fixed-point IIR filter implemented in Task 2. Both Form I and Form II were integrated into the real-time loop similar to the FIR implementation, applying the filter when the loop is disabled.

Performance and Accuracy Results The variable `samples_per_sec` was used to understand if the DSP was able to process all the incoming audio buffers from both channels (left and right).

Table 1.9: Measured real-time samples_per_sec for different optimization levels and IIR filter applied (enable_loop = 0).

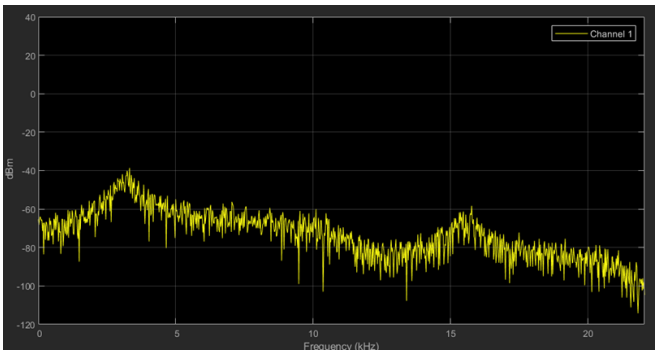
IIR Structure	Optimization Level	samples_per_sec	Real-time
Form I	-O2	48025	Yes
Form II	-O2	48025	Yes
Form I	Disabled	48000	No
Form II	Disabled	47974	No

As we can see from table 1.9, with any compiler optimization the IIR filters in form I and form II were able to achieve 48,000 samples/s, which corresponds to the theoretical expectations. Unlike FIR, both the IIR biquad processing don't exhibit any performance degradation, and no implementation shows a critical dependency on the compiler optimization level.

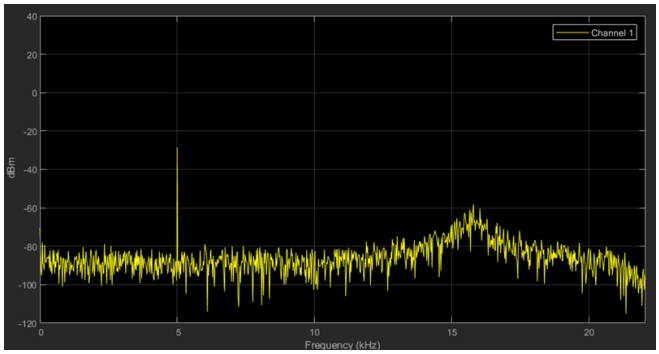
Verification with some audio filters from Matlab In this section, we tested the same signals generated from MATLAB as in section 1.3.1, with the optimization level set to `-O2`, for both IIR form I and IIR form II.

This first part shows IIR with form I:

- **White noise 48k, 30**
  For `enable_loop = 1`, refer to figure 1.3a. The IIR filters implemented are of the second order, since they use biquads. This can be inferred directly from the equation used in form I, where there are two delayed input terms and two delayed output terms. A filter with this structure has a single transition band and a smooth roll–off. Esperimentally we have verified that since, when the IIR filter is active (`enable_loop = 0`), a small peak around 3kHz appears, indicating the second order IIR filter, thus the only frequency that will be cut is in the range around 3kHz.
- **Tone 5k, 30, 48k**
  With enable_loop =1 refer to figure 1.1a. With enable_loop=0, the corresponding peak remains visible in the spectrum and slightly reduced. This is because, as illustrated above in the white noise signal, the filter will cut only the frequencies around 3kHz, thus the tone peak is not in this range.



(a) IIR form I: White noise



(b) IIR form I: Tone

Figure 1.4: IIR Form I spectral analysis (enable_loop=0)

- **Chirp 48k, 12k, 1k**
  For `enable_loop = 1`, refer to figure 1.2a. When the IIR filter is enabled (`enable_loop = 0`), there is almost no visible spectral variation, with no significant attenuation in any frequency region. This is due to the low order of the biquad and its low gain variation over most of the audio band.

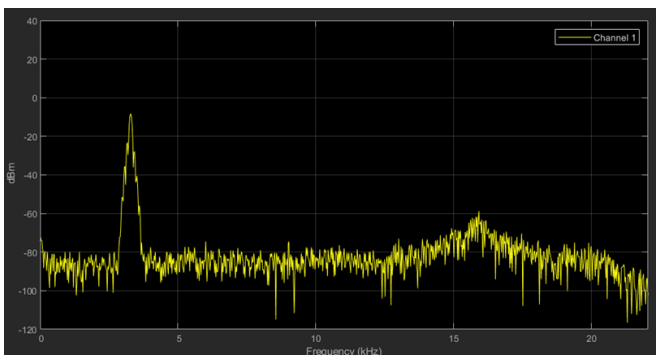
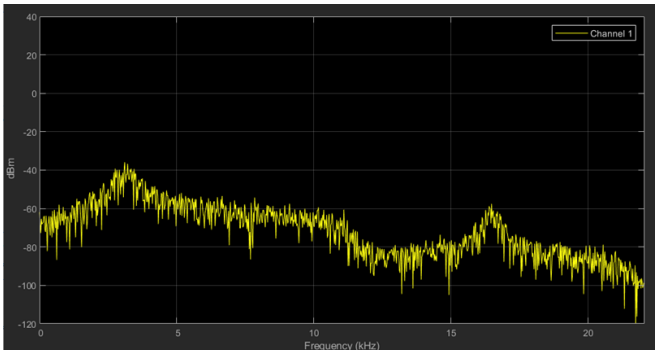


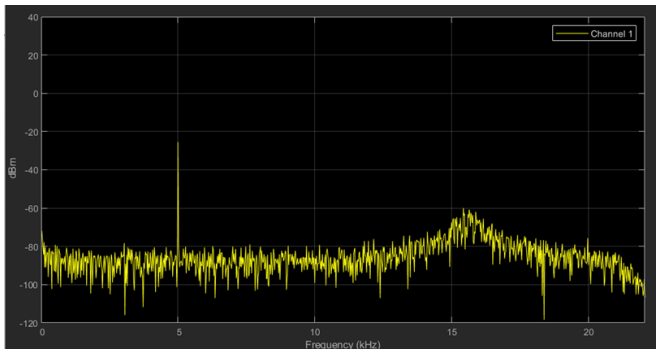
Figure 1.5: IIR form 1 play chirp, enable_loop=0

This second part shows IIR with form II:

- **White noise 48k, 30**
  With enable_loop =1 refer to figure 1.3a. With enable_loop=0, we can almost see the same effect happening in IIR form I.
- **Tone 5k, 30, 48k**
  With enable_loop =1 refer to figure 1.1a. With enable_loop=0, the peak at 5kHz is attenuated of around 20 dB, while the small peak visible at 10kHz disappears. This is coherent with the result found with IIR form I.



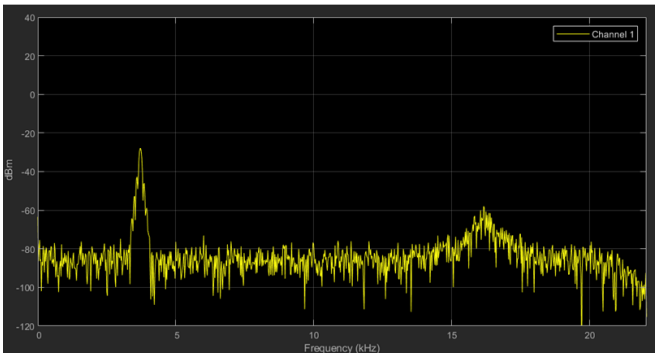
(a) IIR form II: White noise



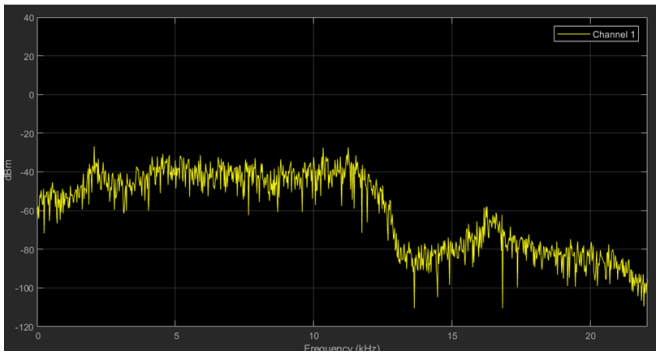
(b) IIR form II: Tone

Figure 1.6: IIR Form II spectral analysis (enable_loop=0)

- **Chirp 48k, 12k, 1k**
  With enable_loop =1 refer to figure 1.2a. With enable_loop=0, we have two different behaviors according to the selected level of sound: with the input volume set to 50% (figure 1.7a), the output spectrum has the same shape of the chirp signal unfiltered, but the its amplitude is reduced by approximately 20 dB, as well as with the IIR form I. When the volume is increased to 100% (figure 1.7b), the spectrum becomes almost flat and the chirp structure is no longer distinguishable. This can be explained by the saturation of the internal states of the biquads, resulting in the generation of an almost uniform spectrum similar to a noise signal. This is the only signal that shows a different behaviour related to the volume level.



(a) Volume 50%



(b) Volume 100%

Figure 1.7: IIR form II play chirp, enable_loop=0

1.4 FIR vs IIR fixed point real time at f=48 kHz

Now, everything we have done in section 1.3.1 and 1.3.2, has been done again by changing the sampling frequency at 48kHz. It's important to highlight that for the IIR implementation, new IIR filters coefficients have Q1.14 representation, thus this has been changed accordingly in the C code.

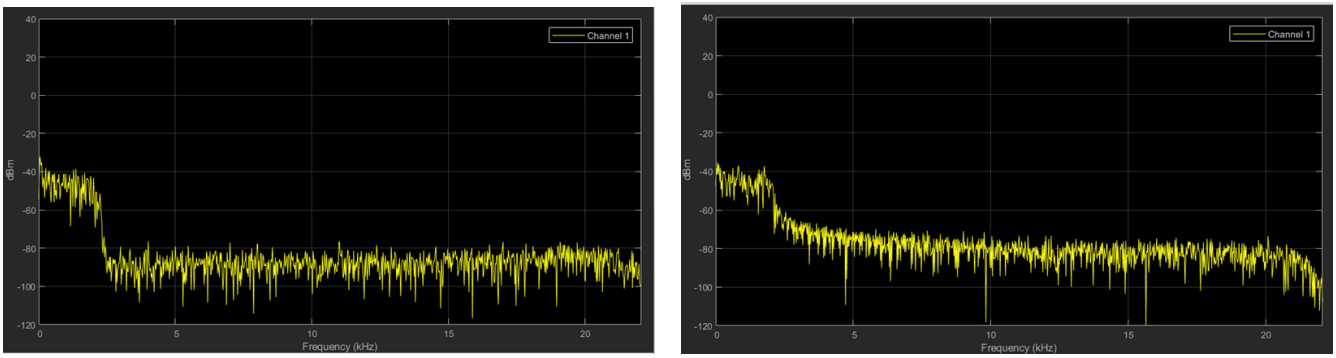
Table 1.10: Measured real-time samples_per_sec for different optimization levels for FIR and IIR filters (enable_loop = 0).

Filter	Optimization Level	samples_per_sec	Real-time
FIR Linear	Disabled	4096	No
FIR Linear	-O0	6707	No
FIR Linear	-O1	6195	No
FIR Linear	-O2 and -O3	76851	No
FIR Circular	Disabled	7065	No
FIR Circular	-O0	12185	No
FIR Circular	-O1	12697	No
FIR Circular	-O2 and -O3	96000	**Yes**
IIR Form I	Disabled	55552	No
IIR Form I	-O0	56115	No
IIR Form I	-O1 -O2	96025	**Yes**
IIR Form II	Disabled	63488	No
IIR Form II	-O0	75878	No
IIR Form II	-O1 and -O2	96000	**Yes**

As we can see from table 1.10, in this new configuration only FIR circular filter with the maximum level of optimization and IIR filters in both forms with optimization level 1 or 2 are able to process the total number of sample_per_second. This is because, when the ADC sampling frequency was increased from 24 kHz to 48 kHz per channel, the total number of samples to be processed per second doubled (from 48,000 to 96,000). Conseguently, the available time to process each block was halved, and only the most optimized and fast implementations are now able to process in real time.

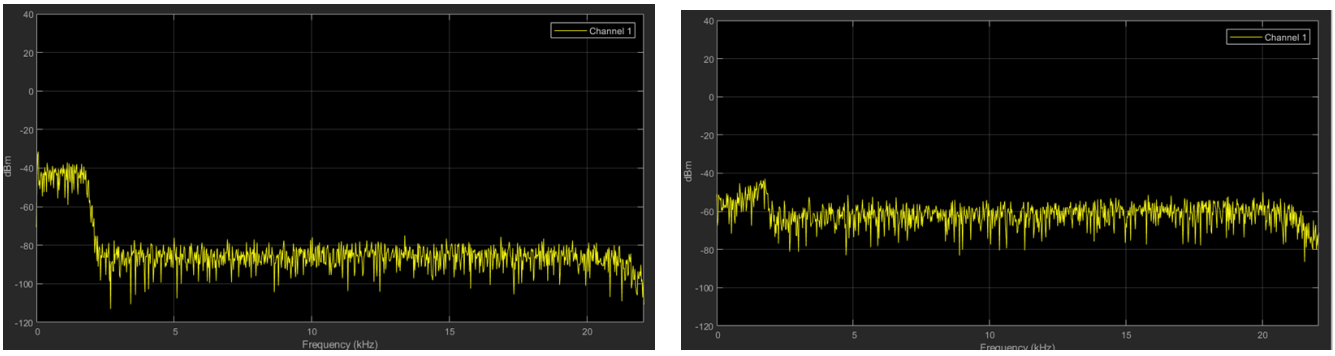
Now we can continue with the analysis of the filters behaviour applied to the previous signals generated by Matlab.

- **White noise 48k, 30**
  Since the DSP sampling rate was increased to 48 kHz, its Nyquist frequency moved from 12 kHz to 24 kHz. Therefore, white noise is now a flat signal up to approximately 24 kHz.



(a) FIR circular optimization -O2 **(b)** FIR linear optimization -O3

Figure 1.8: FIR White Noise Response (48kHz)



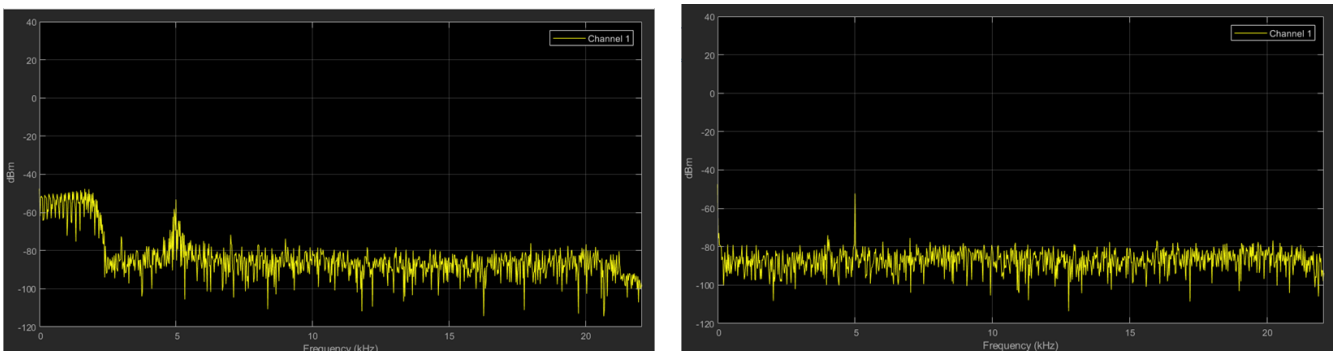
(a) IIR form I optimization -O3 **(b)** IIR form II optimization -O3

Figure 1.9: IIR White Noise Response (48kHz)

From figure 1.8a and 1.8b, both FIR implementations behave as low-pass filters. In particular, 1.8a shows a sharper attenuation immediately above few kHz. The main difference between the two FIR implementations is not about the frequency shaping but rather about the execution speed.

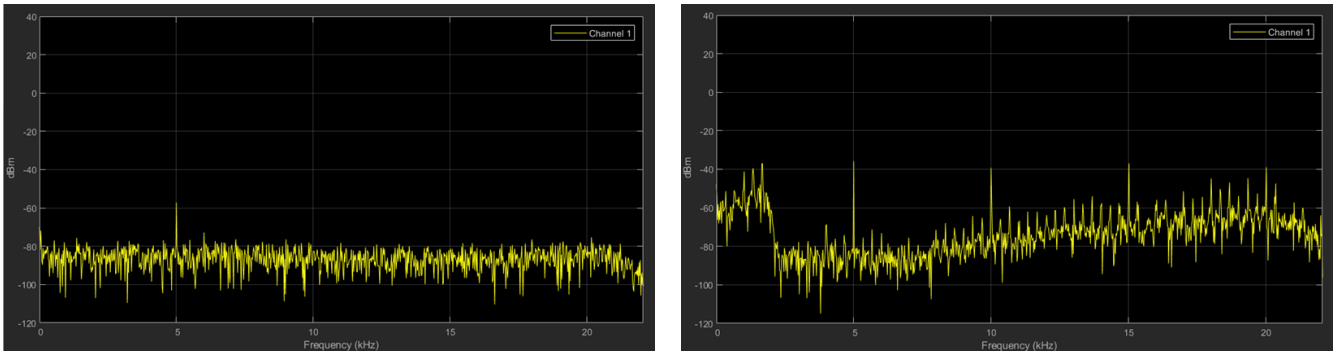
From figure 1.9a and 1.9b, we can still see the same attenuation trend as for the FIR, even if IIR form II seems to be less selective.

- **Tone 5k, 30, 48k**



(a) FIR linear optimization O3 **(b)** FIR circular optimization -O2 (Vol 100%)

Figure 1.10: FIR Filters Tone Response (48kHz)



(a) IIR form II at volume 50% optimization -O3 **(b)** IIR form II at volume 100% optimization -O3

Figure 1.11: IIR Form II Tone Response (48kHz)

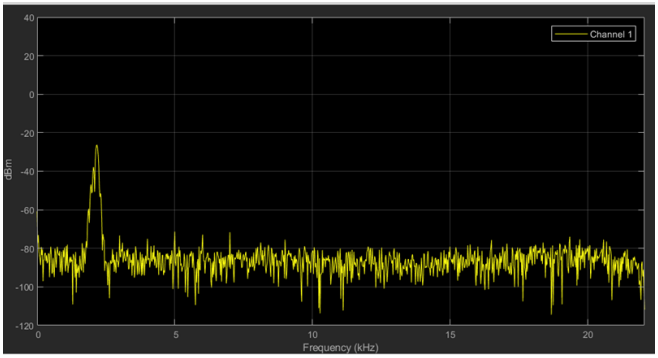
When a 5 kHz tone was used as input, all filters initially showed the presence of multiple thin ”ghost peaks” across the spectrum. They can be explained as a result of saturation, generating a sinusoidal that is no longer perfectly harmonic and produces additional spectral components. After the transient settles only the true 5 kHz tone remains, and this is exactly what has happened during the lab session. This behaviour was most noticeable at high volume levels and almost disappeared when the tone amplitude was reduced.

As we can analyse from figure 1.10a and 1.10b, both FIR filters shows the peak at 5kHz, meaning the filter is not selective in this frequency window. The same behaviour was observed for IIR form I.

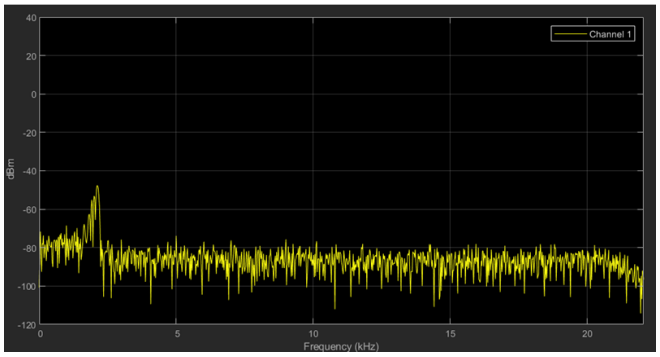
From figure 1.11a we can see clearly the peak at 5kHz, for the same reason explained before, while in figure 1.11b, where the volume was increased at 100%, the output signal is in a saturation mode, the harmonic distortion becomes dominant and the fundamental component is no longer distinguishable.

Overall, the tone signal clearly highlights the instability of the filter based on the volume magnitude.

- **Chirp 48k, 12k, 1k**



(a) FIR circular optimization -O2 at volume 50%

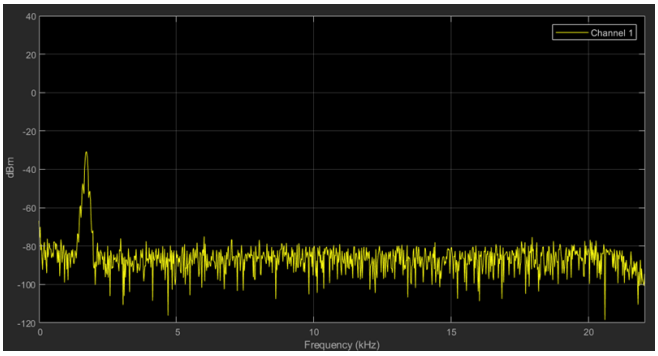


(b) IIR form I optimization -O3 at volume 50%

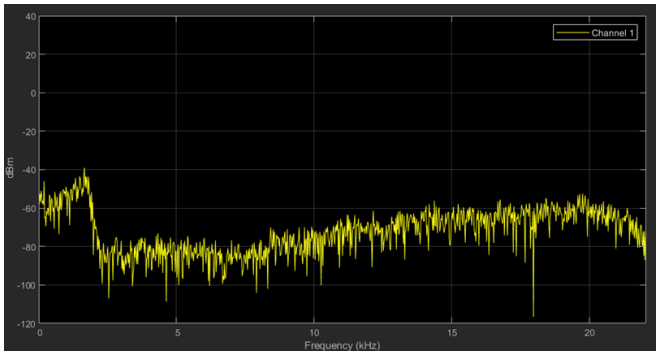
Figure 1.12: FIR circular and IIR form I chirp

From figure 1.12a, we noticed that the chirp was attenuated around 4kHz when the volume was set to 50%, while when the volume was at its maximum there was no evident saturation, rather the same behaviour but with higher magnitude.

From figure 1.12b, at 50% volume, the chirp sweep is visible. When the volume is increased to 100%, the behavior remains unchanged, just with increase magnitude but no saturation occurred. This implies that Form I is numerically stable and handles input amplitude well under the tested conditions.



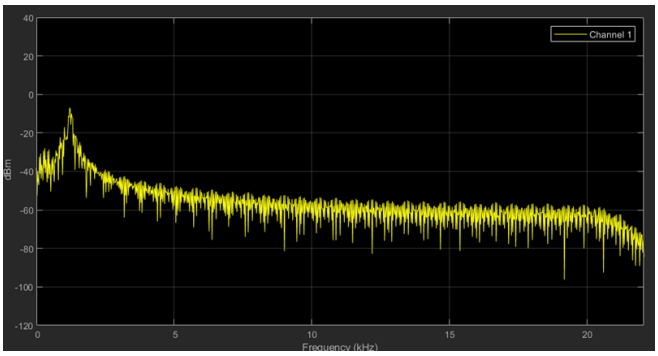
(a) IIR form II optimization -O3 at volume 20%



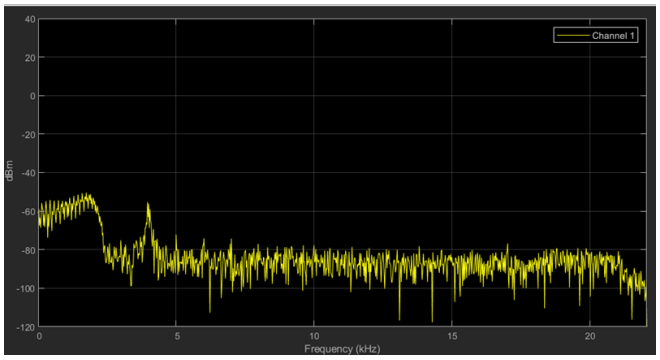
(b) IIR form II optimization -O3 at volume 100%

Figure 1.13: IIR form II chirp

From figure 1.13, we can see that at full volume (Figure 1.13b) the chirp collapses into a flattened spectrum,therefore the filter saturates, and the sweep cannot be distinguished. This was noticed also at a volume of 50%. Only when the volume is reduced to 20% (figure 1.13a) the chirp become clear, with the peak moving clearly as frequency increases. In conclusion, this structure is more sensitive to input amplitude compared to the others.



(a) FIR linear optimization -O3 at volume 100% for the first few seconds



(b) FIR linear optimization -O3 at volume 100% after few seconds

Figure 1.14: FIR linear chirp

As we can notice from figure 1.14a, the filtered signal when the volume is set to 100% has a peak that behaves not as expected, but after few seconds, it becomes more stable and it shifts to the right (figure 1.14b). When the volume was set to 50% only the latter behaviour was noticed, indicating that this effect is due to the implementation itself rather than the input amplitude.

Chapter 2

Laboratory on audio compression

2.1 Task 1: Test of the audio codec

The main task of this section is the implementation and evaluation of a lossless audio codec based on linear prediction and Rice–Golomb entropy coding. The implemented codec follows this scheme:

- **Prediction:** The input signal is estimated using past samples (Polynomial orders 0-2 and FIR order 5).
- **Error Calculation:** The difference between the actual sample and the prediction is computed.
- **Entropy Coding:** The prediction error is encoded using Rice–Golomb codes.
- **Decoding:** The process is reversed to reconstruct the original signal.

Since the same predictor is used at both ends and no quantization is applied, the system is theoretically lossless.

Implementation We chose Rice-Golomb coding because audio prediction errors typically follow a geometric distribution (two-sided), which this coding scheme handles efficiently. The core parameter k determines the split between the unary part (quotient) and the binary part (remainder). We implemented different predictor configurations: no prediction (P0), polynomial predictors of order 1 (P1) and 2 (P2), and a generic FIR predictor of order 5 with Q3.12 coefficients. All four reconstruction errors were verified to be zero, confirming the lossless nature of the implementation.

Performance and Accuracy Results For a fixed input length of 8192 samples, we measured the total encoded bits and the reconstruction error to evaluate compression efficiency.

Predictor	*rec_error* (16 bit)			*rec_error* (8 bit)			*num_bits* (16 bit)			*num_bits* (8 bit)		
	k=5	k=7	k=9	k=5	k=7	k=9	k=5	k=7	k=9	k=5	k=7	k=9
P0	0	0	0	$1.6 * 10^8$ - $1.4 * 10^9$	0	0	37617-934268	11162-289894	4924-141110	37617-infinite	11162-289894	4924-141110
P1	0	0	0	0	0	0	5732-165737	3198-97869	2939-93526	5732-165737	3198-97869	2939-93526
P2	0	0	0	0	0	0	3733-111755	2701-84464	2849-90751	3733-111755	2701-84464	2849-90751
P_FIR	0	0	0	0	0	0	3278-93204	2588-79941	2831-90230	3278-93204	2588-79941	2831-90230

Table 2.1: Different reconstruct errors and number of bits encoded for small buffer (BUFF_LEN*8) and the enlarged one (BUFF_LEN*16), with different k value. Values in the table have been reported after first cycle and after the 32th.

Since the encoder of P0 was highly suboptimal, the size of the code was exceeding the maximum value (8192) of bits. Hence, we increased the buffer size from BUFF_LEN*8 to BUFF_LEN*16.

Afterwards, computational performance is evaluated in terms of CPU clock cycles per processed sample.

Predictor	CPU cycles per encoded sample					
	k = 5		k = 7		k = 9	
	Disabled	O2	Disabled	O2	Disabled	O2
P0	6.66	2.53	4.84	2.08	4.49	1.95
P1	7.10	2.62	5.28	2.17	4.93	2.04
P2	8.41	3.31	6.59	2.86	6.24	2.73
P_FIR	11.95	4.93	10.13	4.48	9.78	4.35

Table 2.2: CPU clock cycles per encoded sample measured on compute_error for different predictors, k values, and compiler optimization levels.

The values contained in table 2.2 have been obtained by summing the average count of CPU clock cycle for encoding and prediction, divided by the total number of samples (=8192).

Analysis of Results Table 2.1 confirms that the implemented codec is strictly lossless, yielding a reconstruction error of zero for all stable configurations. The only exception occurred with the zero-order predictor (P_0) in the initial setup, where the lack of redundancy removal caused data expansion (Rice-Golomb overhead), exceeding the allocated bit buffer size. Increasing the buffer size resolved the issue, restoring lossless behavior. Compression performance shows a direct dependency on predictor complexity: P_0 produces the largest bitstreams, while the 5th-order FIR predictor minimizes the bit rate. Regarding the Rice parameter, $k = 7$ proved to be the optimal trade-off between the unary and binary codeword partitions for the typical error distribution, whereas $k = 9$ was generally suboptimal. Table 2.2 highlights the

trade-off between coding efficiency and computational cost: FIR predictor requires the highest number of CPU cycles per sample due to the accumulation of MAC operations required by the 5th-order filter. Enabling compiler optimization (`-O2`) improved performance, reducing cycle counts by more than 50%. Notably, increasing the Rice parameter k consistently reduces execution time. A larger k results in a smaller quotient q in the Rice-Golomb decomposition. Since q is encoded in unary, minimizing it reduces the number of `PACK_BITS` calls, speeding up the entropy coding stage. Based on the worst-case computational cost measured ($C \approx 11.95$ cycles/sample) and a CPU clock of 100 MHz, the theoretical maximum sampling frequency for real-time mono processing is:

$$f_{s,max} = \frac{f_{clk}}{C} = \frac{100 \text{ MHz}}{11.95} \approx 8.37 \text{ MHz} \tag{2.1}$$

2.2 Task 2: Real-time audio coding

The audio codec was integrated into the real-time loop using the DMA double-buffering mechanism. The primary goal was to evaluate the trade-off between compression efficiency, computational complexity, and real-time stability under different sampling frequencies (f_s) and compiler optimization levels.

2.2.1 Silence and Sine Wave Analysis

An initial validation was performed using 0% input volume to measure the algorithmic overhead. As Table 2.3 shows, the prediction schemes $P1$, $P2$, and P_{FIR} successfully lowered the bitrate to the theoretical minimum (sign bit + overhead $\approx$ 9 bits). In contrast, $P0$ treated the residual thermal noise as unpredictable data and failed to compress it.

Table 2.3: Bitrate (bits/sample) at Volume 0% (Silence).

Predictor	12 kHz	16 kHz	24 kHz	48 kHz
P0	9.5	9.5	9.5	9.5
P1, P2, P_FIR	9	9	9	9

Subsequently, the system was stressed with a 1 kHz full-scale sine wave. Table 2.4 highlights a critical hardware constraint: while $f_s = 48$ kHz theoretically offers the best compression (due to high signal correlation between samples), the P_{FIR} predictor failed real-time operation without optimization. Specifically, with optimization *Disabled*, the processing time per sample exceeded the available time slot ($T_s \approx 20.8\mu s$), causing the effective throughput to drop to $\approx$ 81 kSamples/s (below the required 96 kSamples/s for stereo). Enabling `-O2` optimization was mandatory to fit the 5th-order filter computation within the instruction cycle budget.

Table 2.4: Bitrate (bits/sample) for 1 kHz Sine Wave at Volume 100%. Comparison of Compiler Optimization (-O2).

Predictor	12 kHz		16 kHz		24 kHz		48 kHz	
	Off	**-O2**	**Off**	**-O2**	**Off**	**-O2**	**Off**	**-O2**
P0	25.3	25.2	25.4	25.3	9.5	9.5	25.3	25.3
P1	17.1	17.2	15.0	15.0	13.0	13.0	9.0	9.0
P2	13.0	13.0	11.2	11.2	9.9	9.9	9.0	9.0
P_FIR	11.5	11.6	10.3	10.3	9.0	9.0	**Fail**	**9.0**

2.2.2 Adaptive Rice Parameter on Real Audio

Finally, the static k was replaced with an adaptive function computing the optimal Rice parameter per buffer. Testing with a dynamic audio track (YouTube source) at $f_s = 48$ kHz revealed the impact of signal non-stationarity.

Table 2.5: Adaptive Codec Performance: Real Audio vs. Silence ($f_s = 48$ kHz, -O2).

Predictor	Real Audio (Vol 100%)			Silence (Vol 0%)		
	Bits/Sample	K_L	K_R	**Bits/Sample**	K_L	K_R
P0	13.88	11	11	9.04	6	6
P1	11.72	9	9	4.05	1	1
P2	12.17	9	9	4.98	2	2
P_FIR	12.80	11	11	5.52	3	2

As shown in Table 2.5, real audio yields a higher bitrate ($\approx$ 12.8 bits for P_{FIR}) compared to the pure sine wave ($\approx$ 9 bits). This increase is due to the non-linear components and transients in music, which increase the prediction error entropy. However, the adaptive logic proves critical in silence: the system automatically lowers k to 1 (for P1), achieving $\approx$ 4 bits/sample, drastically outperforming the static $k = 7$ configuration used in previous tasks.